

```
1  # MySQL 数据库设计课件
2
3  > 适用对象：数据库初学者、软件开发初学者
4  > 学习目标：掌握 MySQL 数据库设计的基本思想、表结构设计方法、主键外键、数据类型、范式设计
   以及常见数据库设计流程。
5
6  ---
7
8  ## 一、数据库设计概述
9
10  数据库设计是软件开发中非常重要的环节。一个系统的数据是否清晰、稳定、易扩展，很大程度上取决于数据库设计是否合理。
11
12  MySQL 是一种常见的关系型数据库管理系统，广泛应用于网站、后台管理系统、企业信息系统等项目中。关系型数据库的核心思想是：使用“表”来存储数据，表与表之间通过关系进行连接。
13
14  例如，一个在线教学平台可能包含以下数据：
15
16  - 用户信息
17  - 课程信息
18  - 教师信息
19  - 订单信息
20  - 学习记录
21  - 评论信息
22
23  这些数据不能随意堆在一起，而是要按照业务关系设计成多张表。
24
25  ---
26
27  ## 二、数据库设计的基本流程
28
29  数据库设计一般可以分为以下几个步骤：
30
31  ```text
32  需求分析 → 概念设计 → 逻辑设计 → 物理设计 → 数据库实现 → 优化维护
```

Fence 1

0.1 1. 需求分析

需求分析是数据库设计的第一步，主要目的是弄清楚系统需要存储哪些数据。

例如设计一个学生管理系统，需要分析：

- 系统中有哪些对象？
- 每个对象有哪些属性？
- 对象之间有什么关系？
- 哪些数据需要长期保存？
- 哪些数据需要查询、修改、删除？

学生管理系统中常见对象包括：

- 学生
- 班级

- 教师
- 课程
- 成绩

0.1 2. 概念设计

概念设计主要是从业务角度抽象出实体、属性和关系。

常用工具是 E-R 图。

E-R 图包括：

- 实体：现实中的对象，例如学生、课程
- 属性：实体的特征，例如姓名、年龄
- 关系：实体之间的联系，例如学生选修课程

示例：

```
1 | 学生 -- 选修 -- 课程
```

Fence 2

学生和课程之间是多对多关系，因为一个学生可以选择多门课程，一门课程也可以被多个学生选择。

0.1 3. 逻辑设计

逻辑设计是把概念模型转换成数据库表结构。

例如：

学生表：

```
1 | student(id, name, age, class_id)
```

Fence 3

课程表：

```
1 | course(id, course_name, credit)
```

Fence 4

选课表：

```
1 | student_course(id, student_id, course_id)
```

Fence 5

在逻辑设计阶段，需要确定：

- 表名
- 字段名
- 字段类型
- 主键
- 外键

- 表之间关系

0.1 4. 物理设计

物理设计主要考虑数据库在 MySQL 中如何真正存储。

包括：

- 选择合适的数据类型
- 设置索引
- 设置存储引擎
- 设置字符集
- 设计约束条件
- 考虑性能优化

例如，MySQL 常用存储引擎是 `InnoDB`，它支持事务、外键和行级锁，适合大多数业务系统。

1. 三、表的设计

表是数据库中最基本的存储单位。设计表时，要做到结构清晰、字段合理、关系明确。

1.1 1. 表名设计规范

表名建议使用英文小写单词，多个单词之间用下划线连接。

例如：

```
1  user
2  student
3  course
4  order_info
5  student_score
```

Fence 6

不推荐使用中文表名，也不推荐使用拼音缩写，因为可读性差。

1.2 2. 字段名设计规范

字段名也建议使用英文小写加下划线。

例如：

```
1  user_id
2  user_name
3  create_time
4  update_time
5  phone_number
```

Fence 7

字段名要做到见名知意，不要使用过短、含义不明的名称。

不推荐：

```
1 | a
2 | b
3 | xm
4 | sjh
```

Fence 8

推荐：

```
1 | student_name
2 | phone_number
```

Fence 9

1.3 3. 每张表建议包含的字段

在实际项目中，很多表都会包含以下基础字段：

```
1 | id
2 | create_time
3 | update_time
4 | is_deleted
```

Fence 10

说明：

字段	说明
id	主键
create_time	创建时间
update_time	更新时间
is_deleted	是否逻辑删除

Table 1

示例：

```
1 | CREATE TABLE student (
2 |     id BIGINT PRIMARY KEY AUTO_INCREMENT,
3 |     student_name VARCHAR(50) NOT NULL,
4 |     age INT,
5 |     create_time DATETIME,
6 |     update_time DATETIME,
7 |     is_deleted TINYINT DEFAULT 0
8 | );
```

Fence 11

2. 四、主键设计

主键是表中用来唯一标识一条记录的字段。

例如学生表中：

```
1 | id BIGINT PRIMARY KEY AUTO_INCREMENT
```

Fence 12

表示 `id` 是主键，并且自动增长。

2.1 主键设计原则

1. 主键必须唯一。
2. 主键不应该为空。
3. 主键不应该频繁修改。
4. 主键最好不要使用有业务含义的字段。

例如，身份证号虽然唯一，但不建议直接作为主键，因为它属于业务数据，可能涉及隐私，也可能存在变更或异常情况。

推荐使用无业务含义的自增 `id` 或雪花算法 `id`。

3. 五、外键设计

外键用于表示表与表之间的关系。

例如学生表中有 `class_id` 字段，表示学生属于哪个班级。

班级表：

```
1 | CREATE TABLE class_info (  
2 |     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3 |     class_name VARCHAR(50) NOT NULL  
4 | );
```

Fence 13

学生表：

```
1 | CREATE TABLE student (  
2 |     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3 |     student_name VARCHAR(50) NOT NULL,  
4 |     class_id BIGINT  
5 | );
```

Fence 14

这里的 `class_id` 就表示学生和班级之间的关系。

理论上可以设置外键约束：

```
1 | FOREIGN KEY (class_id) REFERENCES class_info(id)
```

Fence 15

不过在很多互联网项目中，出于性能和灵活性考虑，数据库层面不一定强制设置外键，而是在程序代码中维护数据关系。

4. 六、表之间的关系

数据库表之间常见关系有三种。

4.1 1. 一对一关系

一个用户对应一个用户详情。

```
1 | user -- user_detail
```

Fence 16

用户表：

```
1 | user(id, username, password)
```

Fence 17

用户详情表：

```
1 | user_detail(id, user_id, real_name, address)
```

Fence 18

4.2 2. 一对多关系

一个班级有多个学生，一个学生只属于一个班级。

```
1 | class_info -- student
```

Fence 19

实现方式：

在“多”的一方添加外键。

```
1 | student(class_id)
```

Fence 20

4.3 3. 多对多关系

一个学生可以选择多门课程，一门课程也可以被多个学生选择。

```
1 | student -- course
```

Fence 21

多对多关系需要使用中间表。

```
1 | student_course(id, student_id, course_id)
```

Fence 22

5. 七、MySQL 常用数据类型

5.1 1. 整数类型

类型	说明
TINYINT	小整数
INT	普通整数
BIGINT	大整数

Table 2

常见用法：

1	id BIGINT
2	age INT
3	status TINYINT

Fence 23

TINYINT 常用于状态字段，例如：

1	status TINYINT DEFAULT 1
---	--------------------------

Fence 24

5.2 2. 字符串类型

类型	说明
CHAR	固定长度字符串
VARCHAR	可变长度字符串
TEXT	长文本

Table 3

常用推荐：

1	username VARCHAR(50)
2	phone VARCHAR(20)
3	description TEXT

Fence 25

一般情况下，短文本使用 VARCHAR，长文章内容使用 TEXT。

5.3 3. 时间类型

类型	说明
DATE	日期
TIME	时间
DATETIME	日期和时间
TIMESTAMP	时间戳

Table 4

常用字段：

1	create_time DATETIME
2	update_time DATETIME

Fence 26

5.4 4. 小数类型

涉及金额时，不推荐使用 `FLOAT` 或 `DOUBLE`，因为可能存在精度问题。

推荐使用：

1	DECIMAL(10,2)
---	---------------

Fence 27

例如：

1	price DECIMAL(10,2)
---	---------------------

Fence 28

表示最多 10 位数字，其中小数位 2 位。

6. 八、数据库范式

范式是数据库设计的一种规范，用来减少数据冗余，提高数据一致性。

6.1 1. 第一范式

第一范式要求字段不可再分。

错误示例：

1	联系方式：手机号, 邮箱
---	--------------

Fence 29

正确做法：

1	phone
2	email

6.2 2. 第二范式

第二范式要求表中的非主键字段必须完全依赖主键。

简单理解：

一张表只描述一类事物。

例如学生表只存学生信息，不要同时存课程信息。

错误设计：

```
1 student(id, student_name, course_name, teacher_name)
```

Fence 31

更合理设计：

```
1 student(id, student_name)
2 course(id, course_name, teacher_id)
3 teacher(id, teacher_name)
```

Fence 32

6.3 3. 第三范式

第三范式要求非主键字段之间不能互相依赖。

例如：

```
1 student(id, student_name, class_id, class_name)
```

Fence 33

这里 `class_name` 依赖于 `class_id`，不应该直接放在学生表中。

更合理设计：

```
1 student(id, student_name, class_id)
2 class_info(id, class_name)
```

Fence 34

7. 九、索引设计

索引可以提高查询速度。

例如：

```
1 CREATE INDEX idx_student_name ON student(student_name);
```

Fence 35

适合建立索引的字段：

- 经常作为查询条件的字段

- 经常用于排序的字段
- 经常用于关联查询的字段
- 唯一性要求高的字段

不适合建立索引的字段：

- 数据量很少的表
- 经常修改的字段
- 区分度很低的字段

例如性别字段只有男、女，区分度较低，一般不适合单独建立索引。

8. 十、数据库设计案例：在线教学平台

8.1 1. 用户表

```
1 CREATE TABLE user (  
2     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3     username VARCHAR(50) NOT NULL,  
4     password VARCHAR(100) NOT NULL,  
5     phone VARCHAR(20),  
6     role TINYINT DEFAULT 1,  
7     create_time DATETIME,  
8     update_time DATETIME  
9 );
```

Fence 36

说明：

字段	含义
username	用户名
password	密码
phone	手机号
role	用户角色，1学生，2教师，3管理员

Table 5

8.2 2. 课程表

```
1 CREATE TABLE course (  
2     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3     course_name VARCHAR(100) NOT NULL,  
4     teacher_id BIGINT NOT NULL,  
5     price DECIMAL(10,2) DEFAULT 0.00,  
6     course_desc TEXT,  
7     create_time DATETIME,  
8     update_time DATETIME  
9 );
```

说明：

Fence 37

课程表用于保存课程基本信息，`teacher_id` 表示课程所属教师。

8.3 3. 选课表

```
1 CREATE TABLE user_course (  
2     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3     user_id BIGINT NOT NULL,  
4     course_id BIGINT NOT NULL,  
5     create_time DATETIME  
6 );
```

Fence 38

说明：

用户和课程是多对多关系，所以需要中间表保存选课记录。

8.4 4. 学习记录表

```
1 CREATE TABLE study_record (  
2     id BIGINT PRIMARY KEY AUTO_INCREMENT,  
3     user_id BIGINT NOT NULL,  
4     course_id BIGINT NOT NULL,  
5     progress INT DEFAULT 0,  
6     last_study_time DATETIME  
7 );
```

Fence 39

说明：

`progress` 用来保存学习进度，例如 0 到 100。

9. 十一、数据库设计注意事项

1. 表结构要清晰，不要把所有数据放到一张表中。
 2. 字段类型要合理，避免浪费存储空间。
 3. 金额字段使用 `DECIMAL`。
 4. 每张表最好有主键。
 5. 表名和字段名要规范。
 6. 经常查询的字段可以建立索引。
 7. 不要过度设计，也不要完全不设计。
 8. 重要数据不要物理删除，可以使用逻辑删除。
 9. 密码不能明文存储，应加密后保存。
 10. 数据库设计要结合实际业务需求。
-

10. 十二、课堂总结

MySQL 数据库设计的核心不是简单地创建表，而是根据业务需求合理组织数据。

设计数据库时，要重点考虑：

- 1 数据存什么？
- 2 数据怎么分表？
- 3 表之间什么关系？
- 4 字段类型怎么选？
- 5 主键外键怎么设计？
- 6 查询性能如何保证？
- 7 后期是否方便扩展？

Fence 40

一个优秀的数据库设计应该具备以下特点：

- 结构清晰
- 数据冗余少
- 查询效率高
- 方便维护
- 易于扩展
- 符合业务逻辑

数据库设计是软件开发的基础能力，只有把数据结构设计清楚，后续的 Java 后端、前端页面、接口开发才能更加顺利。

11. 十三、课后练习

11.1 练习一

设计一个图书管理系统数据库，至少包含：

- 图书表
- 用户表
- 借阅记录表

要求写出每张表的字段。

11.2 练习二

设计一个商品订单系统数据库，至少包含：

- 用户表
- 商品表
- 订单表
- 订单详情表

思考订单表和商品表为什么不能直接设计成一张表。

11.3 练习三

根据在线教学平台业务，继续扩展以下表：

- 评论表
- 收藏表
- 课程分类表
- 章节表

要求说明每张表的作用和主要字段。